How the code works

MultiSourceRagChatbot · a guided tour of the codebase · 29 Oct 2026 (state of `main` at `dcf1597`)

The app answers questions using **only your documents**. The guide follows one question through the system: the **UI**, the **API**, the **agent**, then **retrieval** (embeddings, the vector database and reranking), and finally the **LLM calls**. Every section names the exact file, function and line, so you can open it in VS Code (**Ctrl+P**, type the file name, then **Ctrl+G** and the line number).

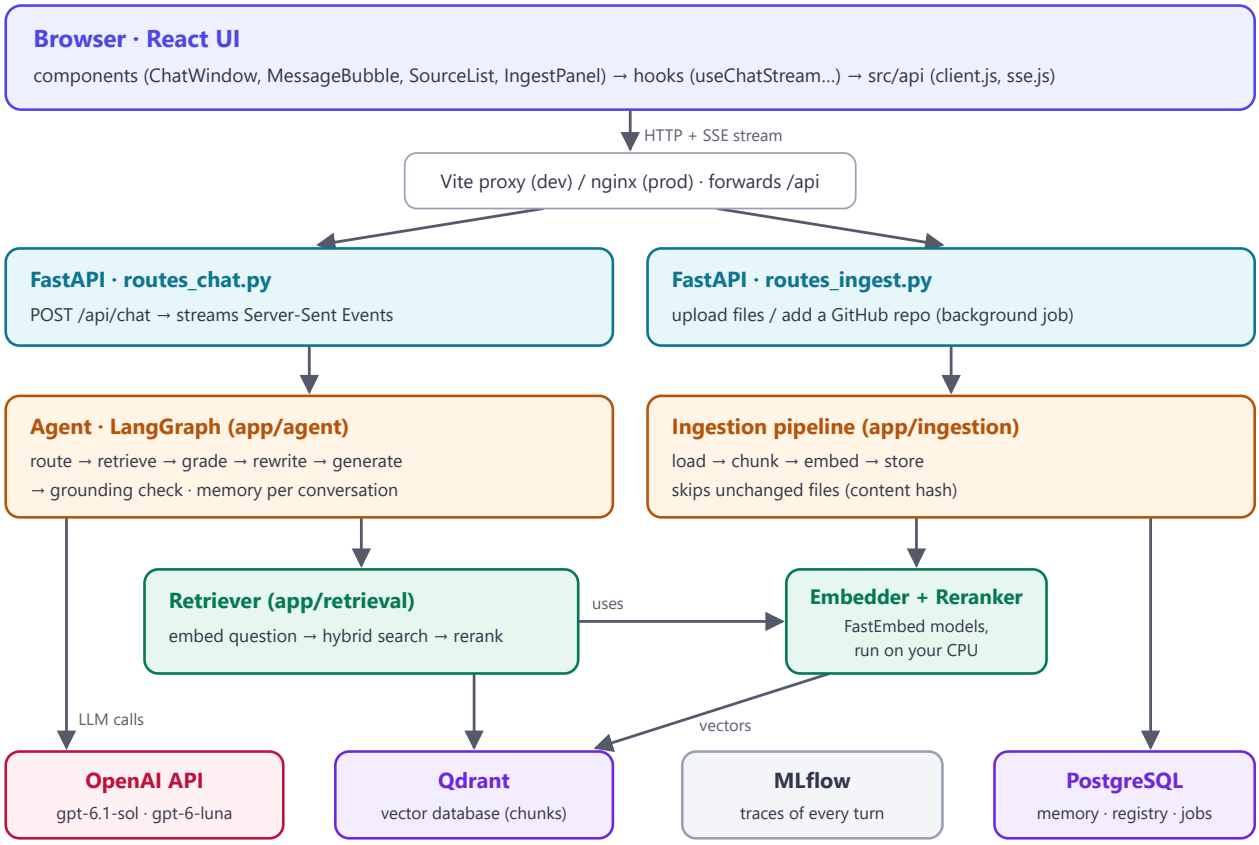


Figure 1: The whole system. Questions flow down the left side; documents flow down the right side. Qdrant, Postgres and MLflow run in Docker (`docker-compose.yml`). OpenAI is the only paid, external service.

UI (React) · API (FastAPI) · Agent / ingestion · Retrieval & embeddings · Databases · LLM (OpenAI)

The journey of one question

Here's what happens when you type "*What does error E-207 mean?*" and press Enter. Each step names the code that does it.

1. **You press Enter.** `ChatInput.jsx` calls `onSend`. `App.jsx` creates a conversation if this is the first message, then `useChatStream.send()` starts.
2. **The browser opens a stream.** `streamChat()` in `src/api/sse.js` sends `POST /api/chat` and reads the answer piece by piece as it arrives.
3. **The API starts the agent.** `chat()` in `app/api/routes_chat.py` checks the conversation exists and isn't busy, then runs the agent in a worker thread (`_run_turn`).
4. **Route** (LLM call, cheap model): is this small talk or a question for the documents? It also rewrites follow-ups like "how do I fix it?" into a full question.
5. **Retrieve** (no LLM): turn the question into vectors (**embeddings**), search **Qdrant** two ways at once (meaning + keywords), merge the two lists, then **rerank** the 10 candidates and keep the best 5.
6. **Grade** (LLM call, cheap model): which of those 5 chunks really help? If none do, **rewrite** the search query and try again (at most 2 times). If still nothing, say "not found" honestly.
7. **Generate** (LLM call, main model, streamed): write the answer from the relevant chunks only, with citations like `[1]`. Each word is sent to the browser as soon as it's written.
8. **Grounding check** (LLM call, cheap model): is every claim supported by the sources? If not, rewrite the answer once. The badge then shows "*Verified against sources*", "*Some claims couldn't be verified*" or "*Not verified*" (the check couldn't run).
9. **Finalize:** save the turn in the conversation's memory (Postgres) and send the `sources` and `done` events (time and cost).
10. **The UI updates live.** Every event becomes a reducer action in `useChatStream.js`. `MessageBubble` shows the steps, the streaming text, the clickable citations, the sources and the badge.

What travels over the stream (Server-Sent Events)

Event	Sent when	What the UI does
`status`	a step starts/finishes ("Searching your documents"…)	adds a line to the progress steps (`StatusSteps.jsx`)
`token`	the model writes a few more characters	appends them to the answer text
`answer_reset`	the grounding check asked for a rewrite	clears the draft answer; the new one streams in
`sources`	the answer is final	shows the numbered sources (`SourceList.jsx`)
`done` / `error`	the turn ended / something failed	shows time, cost and badge, or a readable error message

Why a stream instead of one normal response?
A full answer takes 5–25 seconds. Streaming shows progress and the first words after about a second, so the app feels alive. The Stop button aborts the request, and the server then stops paying for tokens nobody will read (`cancel.set()` in `_run_turn`).

UI · REACT

1 · The user interface (frontend/)

The UI is a React app built with Vite. It never touches a database or OpenAI directly: it only calls the HTTP API under `/api`. The code has three layers, and each layer only talks to the one below it:

Layer	Job	Files
Components	draw things and react to clicks; no network code	`src/App.jsx`, `src/components/*.jsx` (+ `.module.css` styles)
Hooks	hold state (messages, sessions, jobs) and call the API	`src/hooks/useChatStream.js`, `useSessions.js`, `useIngestJobs.js`
API	all `fetch` calls; the stream parser	`src/api/client.js`, `src/api/sse.js`

Where to find each piece on screen

On screen	File : line	Notes
Whole page: sidebar + Chat / Knowledge base view	`src/App.jsx:11`	wires the hooks to the components
Title, input at the top, conversation below	`components/ChatWindow.jsx:14`	example-question chips when empty; auto-scrolls while streaming
Message box + Send / Stop	`components/ChatInput.jsx:11`	Enter sends, Shift+Enter adds a new line
One question + its answer	`components/MessageBubble.jsx:16`	renders Markdown safely (react-markdown), shows cost/time
Progress steps while answering	`components/StatusSteps.jsx:7`	"Searching…", "Checking relevance…"
Numbered sources under an answer	`components/SourceList.jsx:9`	clicking `[1]` in the text opens + highlights source 1
Verified / Unverified badge	`components/GroundingBadge.jsx:13`	result of the grounding check
Conversation list (left)	`components/SessionSidebar.jsx:5`	slides in from the side on phones
Knowledge base tab: upload / GitHub	`components/IngestPanel.jsx:11`	shows documents and job progress
Turning `[1]` into clickable links	`lib/citations.js:13`	`linkCitations()`

The streaming heart of the UI

```
// src/api/sse.js:76  (shortened)
export async function streamChat({ sessionId, message, signal, onEvent }) {
  response = await fetch(`${API_BASE}/chat`, { method: "POST", body: JSON.stringify(…), signal });
  const reader = response.body.getReader();          // read the body as it arrives
  const parser = createSseParser(onEvent);             // sse.js:17 — splits text into events
  for (;;) {
    const { value, done } = await reader.read();
    if (done) break;
    parser.push(decoder.decode(value, { stream: true }));
  }
}
```

```
// src/hooks/useChatStream.js:165  — every event becomes a reducer action
onEvent: ({ event, data }) => {
  if (event === "status") dispatch({ type: "status", step: data.step });
  else if (event === "token") dispatch({ type: "token", text: data.text });
  else if (event === "answer_reset") dispatch({ type: "answer_reset" });
  else if (event === "sources") dispatch({ type: "sources", sources: data.sources });
  …
}
```

Concepts

A

hook

is a reusable function that holds state for components. A

reducer

(`useChatStream.js:16`) is one function that applies every change ("a token arrived") in one place, which makes streaming updates predictable. An

AbortController

lets the Stop button cancel the request. `vite.config.js` forwards `/api` to the Python server on port 8000 during development.

API · FASTAPI

2 · The API (app/api/, app/main.py)

FastAPI turns Python functions into web endpoints. `app/main.py` builds the app (`create_app`). `app/services.py` creates the heavy objects (models, database clients, the agent) once, the first time they're needed, and shares them.

Endpoint	File : line	What it does
`POST /api/chat`	`api/routes_chat.py:116`	answers one message as a Server-Sent Events stream
(helper) run agent in a thread	`api/routes_chat.py:146`	`_run_turn`: worker thread → asyncio queue → stream
`GET/POST/DELETE /api/sessions…`	`api/routes_chat.py:73–97`	list, create, load history, delete conversations
`POST /api/ingest/files`	`api/routes_ingest.py:37`	upload documents → background job
`POST /api/ingest/github`	`api/routes_ingest.py:62`	index a public GitHub repo → background job
`GET /api/jobs`, `/api/documents`	`api/routes_ingest.py:73–88`	job progress and the list of indexed documents
`/health`, `/ready`	`api/routes_health.py`	is the server up? are Qdrant, Postgres and OpenAI reachable?

```
# app/api/routes_chat.py:146 — the bridge between the agent (normal Python) and the stream (async)
def work() -> None:
    for event in agent.stream(message, session_id, cancel):
        loop.call_soon_threadsafe(queue.put_nowait, event)   # hand each event to the web side
…
loop.run_in_executor(None, work)                               # run the agent in a worker thread
while (item := await queue.get()) is not _END:
    yield item                                                 # → becomes one SSE event
```

Why a worker thread?

The agent does slow, blocking work: CPU-heavy embedding and waiting on OpenAI. If that ran on the web server's main loop, every other user would freeze. The thread does the work, and the queue carries events back to the stream as they happen.

Every request and error is logged as JSON (`app/logging_setup.py`). Settings such as model names, limits and URLs are all read in one place: `app/config.py`.

EMBEDDINGS

3 · Getting documents in, and embeddings

Before anything can be searched, documents are **ingested**: read, cut into pieces, turned into numbers and stored.



Figure 2: Ingestion. The whole chain is `IngestionPipeline.ingest_file()` in `app/ingestion/pipeline.py:171`, which calls `_index()`.

Step	File : line	Detail
Read a file	`ingestion/loaders.py:85`	`load_document()` : pdfplumber reads PDFs page by page (keeps page numbers), MarkItDown handles DOCX; size/type checks; detects scanned PDFs
Fetch a repo	`ingestion/github.py:92`	`clone_repo()` : shallow, size-limited clone; code is only *read*, never run
Cut into chunks	`ingestion/chunking.py:43`	`chunk_document()` : splits by headings first, then about 400 tokens per chunk with 75 tokens of overlap so sentences on the border aren't lost
Skip unchanged files	`ingestion/pipeline.py:182`	same content hash as last time → "unchanged", no re-embedding
Embed the chunks	`retrieval/embeddings.py:69`	`embed_passages()` : in batches of 32

What is an embedding?

An **embedding** turns text into a list of numbers. Texts with similar *meaning* get similar numbers, so "the thermostat has no power" lands close to "E-114 low power" even though the words differ. We make **two** kinds for every chunk and every question:

Dense vector: meaning

Model `BAAI/bge-small-en-v1.5`, 384 numbers per text. Good at synonyms and paraphrases. Compared with *cosine similarity* (how closely two vectors point in the same direction).

Sparse vector: keywords

Model `Qdrant/bm25`. Mostly zeros, with one weight per word that appears. Good at exact terms like `E-207` or `URLSafeTimedSerializer`, which meaning-based search can miss.

```
# app/retrieval/embeddings.py:47 — both models are loaded ONCE (slow) and reused
self._dense  = TextEmbedding(settings.embed_dense_model, cache_dir=…)        # bge-small
self._sparse = SparseTextEmbedding(settings.embed_sparse_model, cache_dir=…)  # BM25

def embed_query(self, text):                       # embeddings.py:78 — for the user's question
    dense  = next(iter(self._dense.query_embed(text)))
    sparse = next(iter(self._sparse.query_embed(text)))
    return dense.tolist(), SparseVector(sparse.indices.tolist(), sparse.values.tolist())
```

The models come from FastEmbed and run on your CPU through ONNX: no GPU, no API calls, no cost. They are downloaded once into the model cache.

VECTOR DATABASE · QDRANT

4 · The vector database (app/retrieval/vectorstore.py)

Qdrant stores every chunk as a **point**: an ID, the two vectors, and a **payload** holding the chunk's text, file name, section, page and so on. It runs in Docker on port 6333; its dashboard is at `http://localhost:6333/dashboard`. Everything lives in one **collection** called `knowledge_base`.

Function	File : line	What it does
`VectorStore`	`retrieval/vectorstore.py:62`	the only code that talks to Qdrant
`ensure_collection()`	`retrieval/vectorstore.py:78`	creates the collection: a 384-number cosine "dense" vector + a "sparse" vector with IDF weighting
`upsert_chunks()`	`retrieval/vectorstore.py:100`	writes (or overwrites) one document's chunks
`point_id()`	`retrieval/vectorstore.py:57`	stable IDs from (document, chunk number), so re-indexing overwrites instead of duplicating
`search()`	`retrieval/vectorstore.py:146`	dense, sparse or **hybrid** search (the default)
`_payload()`	`retrieval/vectorstore.py:209`	what is stored next to the vectors (text, source, page…)

Hybrid search with RRF: the key idea

```
# app/retrieval/vectorstore.py:146 — one request, two searches, merged inside Qdrant
kwargs = {
    "prefetch": [
        models.Prefetch(query=dense,        using=DENSE_VECTOR,  limit=limit),  # by meaning
        models.Prefetch(query=sparse_query, using=SPARSE_VECTOR, limit=limit),  # by keywords
    ],
    "query": models.FusionQuery(fusion=models.Fusion.RRF),   # merge the two ranked lists
}
response = self._client.query_points(self.collection, limit=limit, with_payload=True, **kwargs)
```

Why RRF (Reciprocal Rank Fusion)?
The two searches score on different scales: cosine scores sit around 0–1, BM25 scores can be 0–30. They can't simply be added. RRF ignores the scores and uses
positions
instead. Each chunk gets `1/(k + rank)` from each list, and the scores are added. A chunk that ranks well in *both* lists comes out on top.

What else lives in PostgreSQL

Qdrant holds the searchable chunks; Postgres holds everything else:

- **Document registry**: one row per file (name, hash, chunk count), used to skip unchanged files. Tables are in `app/db/schema.sql`, code in `app/db/repositories.py`.
- **Ingest jobs** (`app/db/jobs.py`) and **conversations** (`app/db/sessions.py`).
- **Agent memory**: LangGraph *checkpoints* saved after every step, so a conversation survives a restart (`app/memory/checkpointer.py:26`).

RETRIEVAL

5 · Retrieval: from question to the best 5 chunks

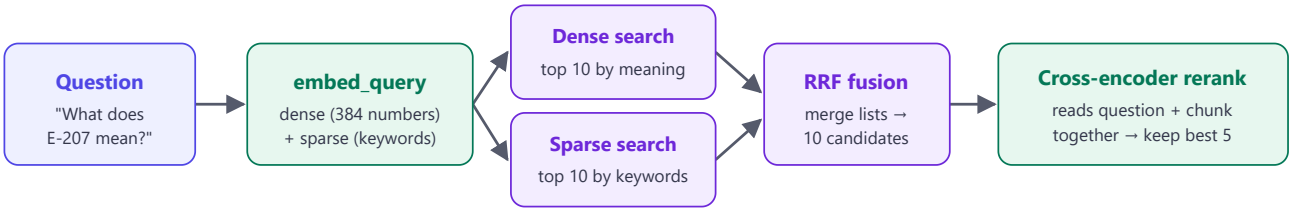


Figure 3: The retrieval funnel. A fast, broad search narrows to 10 candidates; a slower, more precise model picks the best 5. No LLM is used here, so it costs nothing.

Piece	File : line	Detail
`Retriever.retrieve()`	`retrieval/retriever.py:65`	the whole funnel: embed → search → rerank → top 5
`_rerank()`	`retrieval/retriever.py:100`	scores each candidate and sorts best-first
`Reranker.score()`	`retrieval/reranker.py:37`	model `Xenova/ms-marco-MiniLM-L-6-v2` (FastEmbed, CPU)
`with_context()`	`ingestion/models.py:41`	adds file name + section to the chunk text, so "Warranty → Claims" helps both search and reranking
Settings	`config.py:65–66`	`retrieve_candidates = 10`, `rerank_top_k = 5`

```
# app/retrieval/retriever.py:65 (shortened)
dense, sparse = self._embedder.embed_query(question)                  # 1. question → vectors
hits   = self._store.search(dense, sparse, mode="hybrid", limit=10)   # 2. Qdrant hybrid + RRF
chunks = [_to_chunk(hit) for hit in hits]
chunks = _rerank(self._reranker, question, chunks)                    # 3. precise re-ordering
return chunks[:top_k]                                                 # 4. best 5
```

Why two stages (bi-encoder, then cross-encoder)?

The embedding model encodes the question and each chunk *separately*. That's fast, because the chunk vectors are computed once at ingestion, but it's approximate. The

cross-encoder

reads the question and a chunk *together*, which is much more accurate but too slow to run on thousands of chunks. So the fast search picks 10 and the precise model orders them.

AGENT · LANGGRAPH

6 · The agent loop (app/agent/)

A plain RAG app does "search, then answer". Ours is **agentic**: it decides what to do and checks its own work. The flow is a **graph** of steps (*nodes*) joined by arrows (*edges*). Some arrows depend on a decision.

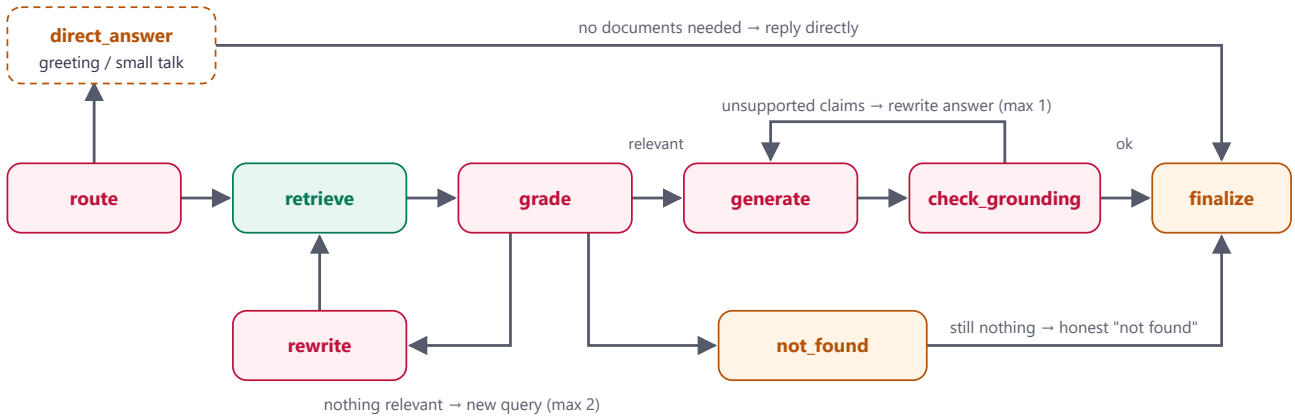


Figure 4: The agent graph, built in `build_graph()` at `app/agent/graph.py:36`. Red nodes call the LLM; the green node is the free retrieval. The loops are **bounded** (`max_query_rewrites = 2`, `max_regenerations = 1`), so the agent can't loop forever or run up the bill.

Piece	File : line	Detail
The graph wiring	`agent/graph.py:36`	`add_node` / `add_edge` / `add_conditional_edges`
The node functions	`agent/nodes.py:45–114`	route, retrieve, grade, rewrite, generate, check_grounding, not_found, direct_answer, finalize
The decisions on the arrows	`agent/nodes.py:128–139`	`after_route`, `after_grade`, `after_grounding`
Shared state (the "notebook")	`agent/state.py:28`	`AgentState`: question, chunks, answer, counters, messages…
Running a turn + streaming	`agent/graph.py:123`	`RagAgent.stream()`; `_events_for()` (line 167) turns graph updates into UI events
Memory	`memory/checkpointer.py:26`	state saved to Postgres per conversation (`thread_id`)

```
# app/agent/nodes.py:131 — the decision after grading
def after_grade(self, state):
    if state.get("relevant"):
        return "generate"                                   # good chunks → write the answer
    if state.get("rewrites", 0) < self.d.settings.max_query_rewrites:
        return "rewrite"                                    # try a better search query
    return "not_found"                                      # be honest instead of guessing
```

LLM CALLS · OPENAI

7 · The LLM calls (app/agent/llm.py, judge.py, prompts.py)

OpenAI is called only from the agent, and every model is created in one function: `make_chat_model()` (`app/agent/llm.py:41`). There are two **roles**. A cheap, fast **judge** makes the yes/no-style decisions, and a stronger **generator** writes the answers.

Call	Model	Code	Returns
route	judge: `gpt-6-luna`, effort none	`agent/judge.py:56`	`RouteDecision`: "retrieve" or "direct" + a standalone question
grade	judge	`agent/judge.py:64`	`GradeResult`: numbers of the relevant chunks
rewrite	judge	`agent/judge.py:73`	`RewrittenQuery`: a better search query
generate	generator: `gpt-6.1-sol`, effort low, *streaming*	`agent/nodes.py:75`	the answer text with `[n]` citations
check_grounding	judge	`agent/judge.py:79`	`GroundingVerdict`: grounded? + unsupported claims
direct_answer	generator	`agent/nodes.py:109`	a short reply to greetings / small talk

```
# app/agent/llm.py:41 — every cost and safety knob comes from Settings (app/config.py:40-51)
return ChatOpenAI(
    model=model, reasoning_effort=effort,
    max_tokens=settings.llm_max_output_tokens,    # 1024: hard cap on output, and so on cost
    timeout=settings.llm_request_timeout_s,       # 60 s: never hang forever
    max_retries=settings.llm_max_retries,         # 2: retry rate limits / server errors
    streaming=streaming, stream_usage=True,       # count tokens even while streaming
)
```

Structured output: the judge answers in a fixed format

```
# app/agent/judge.py:87 — one helper used by every judge call
def _ask(self, schema, system, user):
    try:
        structured = self._model.with_structured_output(schema, strict=True)   # JSON matching a
Pydantic class
        result = structured.invoke([SystemMessage(system), HumanMessage(user)])
    except Exception as exc:                  # network, rate limit, bad output…
        log.warning("judge_failed", …)
        return None                           # → the caller uses a safe fallback
    return result if isinstance(result, schema) else None
```

Why structured output and fallbacks?

The agent's next step depends on the judge's answer. Free text like "Yes, I think passages 1 and 3…" is hard to parse reliably. With `strict=True`, OpenAI must return JSON matching the schema in `app/agent/state.py:51-64`. If a call still fails, the agent never crashes: grading keeps all chunks, routing falls back to "search", and grounding shows "Not verified".

Also good to know	Where
All prompts (instructions to the models), incl. prompt-injection warnings	`agent/prompts.py:12–82`
Putting the chunks into the prompt, within a token budget	`agent/rag.py:56 (fit_to_budget), :71 (build_messages)`
Cost estimate per answer (the "$0.0017" in the UI)	`agent/llm.py:62 (estimate_cost_usd)`
Tokens + cost logged for every turn	`agent/graph.py (_log_turn)`
Each turn traced step by step in MLflow (http://localhost:5000)	`observability/tracing.py:23`

Typical cost: about $0.001–0.005 per question. The generator writes most of the tokens, and the judge calls (usually three, four if a query is rewritten) together cost a fraction of a cent.

Cheat sheet: where do I look when...

I want to...	Open
change how the chat screen looks	`frontend/src/components/ChatWindow.jsx` + `ChatWindow.module.css`
change colours / fonts everywhere	`frontend/src/index.css` (light and dark tokens)
change how an answer bubble looks	`frontend/src/components/MessageBubble.jsx`
add a new API call in the UI	`frontend/src/api/client.js`, then use it from a hook
change chunk size or overlap	`.env` → `CHUNK_SIZE_TOKENS`, `CHUNK_OVERLAP_TOKENS` (defaults in `app/config.py:69–70`), then re-ingest
get more / fewer chunks per answer	`RETRIEVE_CANDIDATES`, `RERANK_TOP_K` (`app/config.py:65–66`)
change what the model is told	`app/agent/prompts.py`
change the agent's steps or loops	`app/agent/graph.py` (wiring), `app/agent/nodes.py` (steps)
change models (ask the owner first: affects cost)	`.env` → `LLM_GENERATOR_MODEL`, `LLM_JUDGE_MODEL`
support a new file type	`app/ingestion/loaders.py`
see what Qdrant stored	http://localhost:6333/dashboard → collection `knowledge_base`
see why an answer came out the way it did	MLflow traces at http://localhost:5000
see how a file gets chunked (stores nothing)	`python -m app.ingestion.preview <file>`

Tests that show each part working

Part	Tests
UI components, SSE parser, hooks	`frontend/src/**/*.test.jsx|js` (Vitest; `npm test`)
Whole app in a real browser	`frontend/e2e/*.spec.js` (Playwright; `npm run e2e`)
Chunking, fusion, routing logic, prompts (no network)	`tests/unit/` (`pytest`)
Real Qdrant / Postgres / OpenAI	`tests/integration/` (`pytest -m integration`)
All of the above on every push to `main`	`.github/workflows/ci.yml` (GitHub Actions)

Line numbers are for commit `dcf1597`. If code moves later, search for the function name instead (**Ctrl+Shift+F** in VS Code).